

Technical Requirements Document (TRD) v2.0

CloudWise AI

Version: 2.0

Status: Active Development

Last Updated: June 2026

1. Introduction

Purpose

This document defines the technical architecture, infrastructure requirements, system components, integrations, data flow, and engineering standards required for CloudWise AI.

It serves as the primary technical reference for development, deployment, testing, and future scaling.

2. System Overview

CloudWise AI is a cloud infrastructure intelligence platform that connects to AWS or LocalStack environments, discovers resources, analyzes costs, generates optimization recommendations, and provides actionable insights through a modern web dashboard.

3. Technology Stack

Frontend

Framework:

- React 19
- TypeScript
- Vite

Libraries:

- React Router
- React Query
- Recharts

- Axios

Styling:

- Vanilla CSS
 - CSS Variables
 - Responsive Layout System
-

Backend

Framework:

- FastAPI

Language:

- Python 3.12+

Key Libraries:

- SQLAlchemy
 - Alembic
 - Boto3
 - Pydantic
 - Passlib
 - Python-JOSE
-

Database

Development:

- SQLite

Production:

- PostgreSQL

ORM:

- SQLAlchemy

Migration Tool:

- Alembic
-

Cloud Services

Provider:

- AWS

SDK:

- Boto3

Supported Services:

- EC2
- EBS

Planned:

- S3
 - RDS
 - Lambda
-

Local Development

LocalStack

Endpoint:

<http://localhost:4566>

Purpose:

- Offline AWS simulation
 - Demo environment
 - Development testing
-

Hosting

Frontend:

Vercel

Backend:

Render

Database:

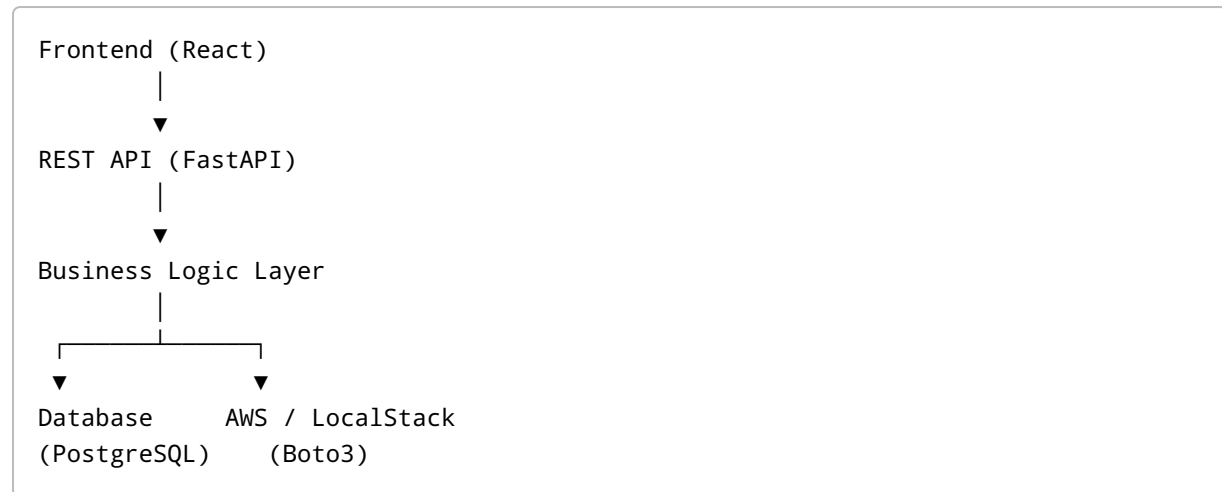
Render PostgreSQL

Optional Cache:

Redis / Upstash

4. System Architecture

High-Level Architecture



5. Core Modules

Authentication Module

Responsibilities:

- User Registration
- Login
- JWT Creation
- Token Validation
- Password Reset

Endpoints:

```
POST /auth/register
POST /auth/login
POST /auth/forgot-password
POST /auth/reset-password
```

Cloud Integration Module

Responsibilities:

- AWS Validation
- LocalStack Validation
- Session Creation
- Resource Discovery

Files:

```
backend/app/services/aws_service.py
```

Functions:

```
connect_aws()
discover_ec2()
discover_ebs()
get_cost_data()
get_cloudwatch_metrics()
```

Resource Discovery Module

Responsibilities:

- Discover infrastructure resources
- Normalize cloud data
- Store inventory

Supported Resources:

EC2

Collected Fields:

- Instance ID
 - Instance Type
 - State
 - Region
 - Tags
-

EBS

Collected Fields:

- Volume ID
 - Size
 - State
 - Attachments
 - Encryption
-

6. LocalStack Architecture

Toggle Flow

Frontend

↓

`use_localstack = true`

↓

Backend API

↓

`connect_aws()`

↓

`endpoint_url = http://localhost:4566`



LocalStack Services

Endpoint Switching Logic

AWS:

```
endpoint_url = None
```

LocalStack:

```
endpoint_url = "http://localhost:4566"
```

Applied To:

- STS
 - EC2
 - CloudWatch
 - Cost Explorer
-

7. Database Architecture

Core Tables

users

Purpose:

Authentication

Fields:

```
id
email
hashed_password
created_at
updated_at
```

cloud_accounts

Purpose:

Store cloud connections

Fields:

```
id
user_id
account_id
provider
region
use_localstack
created_at
```

resource_inventory

Purpose:

Store discovered resources

Fields:

```
id
resource_id
resource_name
resource_type
region
status
arn
tags
metadata_json
created_at
```

recommendations

Purpose:

Store optimization suggestions

Fields:

```
id
resource_id
recommendation_type
priority
estimated_savings
description
created_at
```

metrics

Purpose:

Store utilization data

Fields:

```
id
resource_id
metric_name
metric_value
timestamp
```

8. API Architecture

Authentication APIs

```
POST /auth/register
POST /auth/login
POST /auth/forgot-password
POST /auth/reset-password
```

Cloud APIs

POST /cloud/connect
POST /cloud/scan
GET /cloud/resources

Dashboard APIs

GET /dashboard
GET /dashboard/summary
GET /dashboard/costs
GET /dashboard/health-score

Recommendation APIs

GET /recommendations
GET /recommendations/{id}

Forecast APIs

GET /forecast

Report APIs

POST /reports/generate
GET /reports

9. Recommendation Engine

EC2 Rules

Idle Instance

Condition:

CPU < 5%

Output:

Recommend shutdown

Underutilized Instance

Condition:

CPU < 20%

Output:

Recommend downsizing

EBS Rules

Unattached Volume

Condition:

No active attachment

Output:

Recommend deletion

10. Cost Analysis Engine

Current Source:

AWS Cost Explorer

Responsibilities:

- Daily cost calculation
- Monthly cost calculation
- Service-based grouping

Outputs:

Current Spend
Estimated Spend
Potential Savings

11. Dashboard Engine

Data Sources:

- Resource Inventory
- Metrics
- Recommendations
- Cost Data

KPIs:

- Total Resources
- Monthly Spend
- Potential Savings
- Health Score

Charts:

- Cost Trend
 - Resource Distribution
-

12. Security Requirements

Authentication:

- JWT Tokens
- Password Hashing
- Protected Routes

Credential Protection:

- Never store AWS secrets in plaintext
- Encrypt sensitive values

API Security:

- Input validation
- Request sanitization
- Rate limiting (future)

Transport Security:

- HTTPS only in production
-

13. Logging Requirements

Required Logs:

Authentication:

```
User Login
Failed Login
Password Reset
```

Cloud:

```
AWS Connection
LocalStack Connection
Resource Discovery
```

Infrastructure:

EC2 Discovery Count
EBS Discovery Count
Resources Saved
Dashboard Requests

Errors:

AWS Failures
Database Errors
API Exceptions

14. Performance Requirements

Dashboard Load:

< 3 seconds

Cloud Scan:

< 60 seconds

API Response:

< 500ms average

15. Deployment Requirements

Frontend:

Vercel

Backend:

Render

Database:

Render PostgreSQL

Environment Variables:

```
DATABASE_URL=  
JWT_SECRET=  
AWS_REGION=  
LOCALSTACK_ENDPOINT=http://localhost:4566  
DEMO_MODE=
```

16. Future Technical Roadmap

Version 2.1

- Forecasting Engine
- PDF Reports
- Improved Health Score

Version 3.0

- AI Copilot
- S3 Discovery
- RDS Discovery
- Lambda Discovery

Version 4.0

- Azure Integration
- GCP Integration
- Multi-Cloud Architecture
- Redis Caching
- RBAC

Approval

Document Status:

Approved

Version:

2.0